

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	ATHIN SIVA.S	Reg. No.:	192524237
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



SIMATS Online Programming Lab
DAA PROGRAMMING



CO5 AT1-Constraint-Based Problem Solving
OPEN
Completed
Review →

1/1 answered

View Rubric

Criteria	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Problem Understanding 25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis 25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development 25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts 15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification 10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Problem Statement

The N-Queens problem is a classical constraint-satisfaction problem in which N queens must be placed on an $N \times N$ chessboard such that no two queens attack each other. Since a queen can move horizontally, vertically, and diagonally, every placement must satisfy row, column, and diagonal constraints.

The objective is to develop a backtracking algorithm that places queens one row at a time, checks whether every placement is valid, and immediately abandons a partial configuration when it cannot lead to a solution. This pruning significantly reduces the search space compared with checking every possible arrangement.

2. Constraints

Constraint	Condition	Reason
Row constraint	Exactly one queen is placed in each row.	Prevents two queens from sharing a row.
Column constraint	No two queens may occupy the same column.	Queens attack vertically.
Main diagonal	No two queens may have the same row – column value.	Queens attack along one diagonal direction.
Anti-diagonal	No two queens may have the same row + column value.	Queens attack along the opposite diagonal direction.

3. Representation of the Board

The board can be represented using an array where index i represents a row and the stored value represents the column containing the queen. For example, $[1, 3, 0, 2]$ for $N = 4$ means that queens are placed at (row 0, column 1), (row 1, column 3), (row 2, column 0), and (row 3, column 2).

For efficient constraint checking, three sets are maintained: columns for occupied columns, diag1 for occupied values of row – column, and diag2 for occupied values of row + column.

4. Why Backtracking is Used

A brute-force method could try all possible placements of N queens. If one queen is selected from every row, there are N^N possible column assignments before considering conflicts. Backtracking avoids exploring many of these assignments.

The algorithm builds a solution incrementally. When a queen is placed in the current row, the next row is processed. If no column is valid for the current row, the algorithm goes back to the previous row and moves that queen to another possible column. This process continues until a complete valid configuration is found or all possibilities have been exhausted.

5. Backtracking Algorithm

1. Start with row 0.
2. Try each column from 0 to $N - 1$.
3. Check whether the selected column and both diagonals are free.
4. If the position is safe, place the queen and mark the column and diagonals as occupied.
5. Recursively attempt to place a queen in the next row.
6. If the recursive call succeeds, keep the placement.
7. If it fails, remove the queen and unmark the column and diagonals. This is the backtracking step.
8. If all N rows are successfully filled, a valid configuration has been found.

6. Pruning Technique

Pruning means stopping the exploration of a branch as soon as it violates a constraint. For example, if a queen is already present in column 2, every future attempt to place a queen in column 2 is rejected immediately. Similarly, if $\text{row} - \text{column}$ or $\text{row} + \text{column}$ is already present, the position is rejected because it lies on an occupied diagonal.

This early rejection is important because an invalid partial board can never become a valid complete board. Therefore, there is no reason to continue exploring that branch.

7. Python Program

```
def solve_n_queens(n):
    board = [-1] * n
    columns = set()
    diag1 = set()
    diag2 = set()
    solutions = []

    def backtrack(row):
        if row == n:
            solutions.append(board.copy())
            return

        for col in range(n):
            if col in columns:
                continue
            if row - col in diag1:
                continue
            if row + col in diag2:
                continue
```

```

board[row] = col
columns.add(col)
diag1.add(row - col)
diag2.add(row + col)

backtrack(row + 1)

board[row] = -1
columns.remove(col)
diag1.remove(row - col)
diag2.remove(row + col)

```

```

backtrack(0)
return solutions

```

```

n = int(input("Enter N: "))
solutions = solve_n_queens(n)

print("Number of solutions =", len(solutions))

for solution in solutions:
    for col in solution:
        print("." * col + "Q" + "." * (n - col - 1))
    print()

```

8. Sample Input

4

9. Sample Output

Enter N: 4

Number of solutions = 2

```

.Q..
...Q
Q...
..Q.

```

```

..Q.
Q...
...Q
.Q..

```

10. Explanation of the Sample

For N = 4, the algorithm finds two valid configurations. One configuration can be represented by [1, 3, 0, 2]. The first queen is placed at row 0, column 1. The second is placed at row 1, column 3. The third is placed at row 2, column 0, and the fourth at row 3, column 2.

No two queens share a column, and their row - column and row + column values are also distinct. Therefore, none of the queens attack another queen.

11. How the Search Space is Reduced

Without pruning, selecting a column independently for every row gives N^N possible assignments. The backtracking algorithm immediately rejects a column when it conflicts with an already placed queen. Therefore, the algorithm does not generate complete boards for most invalid partial configurations.

The row constraint itself also reduces the problem because the algorithm places exactly one queen per row. If column conflicts are considered as well, the possible placements are bounded by $N!$ permutations rather than N^N assignments. Diagonal pruning reduces the search even further in practice.

12. Correctness of the Algorithm

The algorithm is correct because a queen is placed in a row only after verifying all three relevant conflict conditions: column, main diagonal, and anti-diagonal. Thus every partial configuration maintained by the algorithm is valid.

When the recursion reaches row N , all N rows contain one queen and every placement was previously validated. Hence the resulting board is a valid N -Queens configuration.

The algorithm also finds all valid configurations because for every row it systematically tries every column that is not pruned. A branch is discarded only when it violates a constraint. Therefore, no valid configuration is removed by the pruning rules.

13. Complexity Analysis

Measure	Complexity	Explanation
Worst-case time	$O(N!)$	At most N choices for the first row, then fewer column choices for subsequent rows; diagonal checks prune additional branches.
Common upper bound	$O(N \cdot N!)$	If board checking/copying overhead is included in some implementations.
Space	$O(N)$	Board, recursion stack, and constraint sets require linear auxiliary space, excluding storage for all solutions.
All solutions	Output-dependent	Storing every solution requires additional space proportional to the number of solutions times N .

The N -Queens problem has exponential/factorial-scale search behavior in the worst case. Backtracking does not change the fundamental difficulty of the problem, but pruning can make the practical running time much smaller than naive exhaustive enumeration.

14. Advantages of Backtracking

Simple recursive formulation.

Rejects invalid placements as early as possible.

Uses less memory than storing all partial boards.

Can find one solution or all solutions.

Naturally supports constraint-based problems such as N-Queens, Sudoku, and graph coloring.

The three-set representation makes safety checking efficient.

15. Limitations

The search space still grows rapidly as N increases.

Large N values may require significant computation time.

If all solutions are stored, output size can become very large.

Backtracking is generally unsuitable for very large instances without additional optimization or specialized techniques.

16. DAA Perspective

From a Design and Analysis of Algorithms perspective, N-Queens demonstrates state-space search, constraint checking, recursion, backtracking, and pruning. The important design principle is to construct a candidate solution step-by-step and abandon a partial candidate as soon as it becomes impossible to complete.

The efficiency comes mainly from avoiding unnecessary states rather than from eliminating the worst-case exponential nature of the problem. This illustrates why pruning is an important optimization technique in combinatorial search problems.

17. Conclusion

The N-Queens problem can be efficiently approached using backtracking. The algorithm places one queen per row and checks column and diagonal constraints before continuing. If a placement is invalid, it is pruned immediately, preventing further exploration of that branch.

The approach is complete because every non-pruned valid placement is explored, so all valid configurations can be generated. Although the worst-case complexity remains factorial/exponential, constraint-based pruning greatly reduces the practical search space. Thus, N-Queens is a clear example of how intelligent search and pruning improve the performance of combinatorial algorithms.